

Analyse métier et DDD — CanBankX

Auteur : Pascal Bourgoin **Date :** Mars 2026 **Projet :** LOG430 — Architecture logicielle

Table des matières

- 1. Contexte métier
- 2. Périmètre fonctionnel
- 3. Langage ubiquitaire
- 4. Bounded Contexts
- 5. Modèle de domaine
- 6. Règles métier
- 7. Cas d'utilisation détaillés

1. Contexte métier

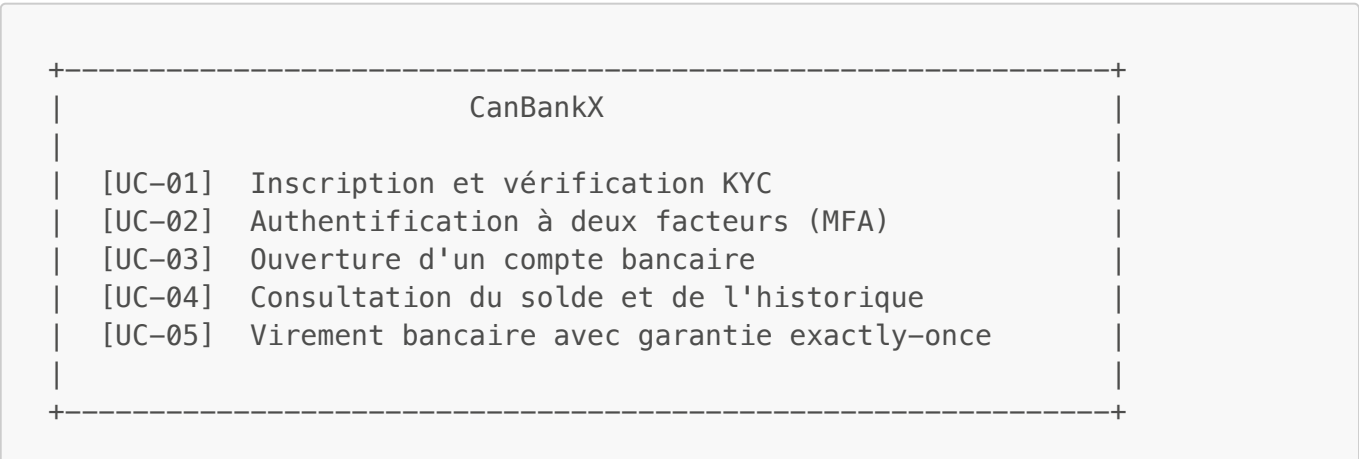
CanBankX est une banque en ligne destinée aux investisseurs particuliers. Les clients peuvent ouvrir des comptes, consulter leurs soldes et effectuer des virements depuis n'importe quel client HTTP. La plateforme est accessible 24h/24 et doit garantir qu'aucun paiement n'est exécuté deux fois, même en cas d'instabilité réseau.

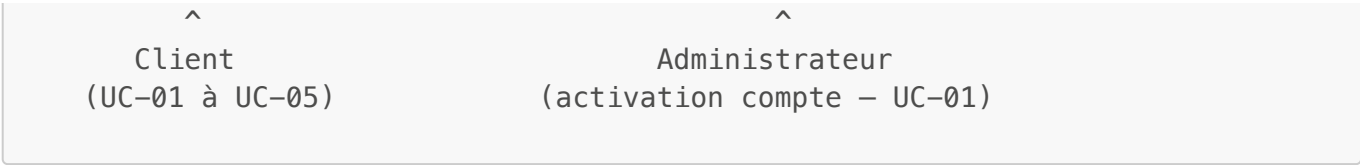
Le système s'adresse à trois profils :

Profil	Besoin principal
Client bancaire	Ouvrir un compte, consulter son solde, effectuer des virements fiables
Développeur	API documentée (Swagger), collection Postman, erreurs normalisées
Ops / Correcteur	Observabilité complète, charge testée, architecture traçable

2. Périmètre fonctionnel

Le projet implémente **5 cas d'utilisation** couvrant le parcours complet d'un client, de l'inscription au premier virement.





Sont **hors périmètre** pour cette phase : trading d'actifs, prêts, cartes de crédit, virements internationaux SWIFT.

3. Langage ubiquitaire

Ce glossaire est partagé entre le code, la documentation et les tests. Les termes ci-dessous ont une définition précise et stable dans tout le projet.

Terme	Définition dans CanBankX
Client	Personne physique inscrite. Possède un cycle de vie : PENDING → ACTIVE → SUSPENDED . Un client inactif (PENDING) ne peut pas ouvrir de compte.
NAS	Numéro d'assurance sociale canadien. Exactement 9 chiffres. Identifiant unique par client — contrainte UNIQUE en base.
OTP	Code numérique à 6 chiffres à usage unique, valide 5 minutes. Envoyé par email lors de l'inscription (KYC) et lors du login MFA.
Challenge Token	Jeton opaque généré à l'étape login (MFA étape 1). Lié à une session de login en cours. Expiré automatiquement par Redis (TTL 5 min).
Account	Compte bancaire appartenant à un Client . Types : CHECKING (chèque) ou SAVINGS (épargne). Identifié par un accountNumber généré aléatoirement.
BankTransaction	Mouvement financier atomique. Types : DEBIT (retrait du compte source), CREDIT (dépôt), TRANSFER (débit source + crédit destination).
Idempotency-Key	Clé fournie par le client HTTP dans le header de chaque requête de paiement. Garantit qu'un réseau instable ne provoque pas de double débit.
AuditLog	Enregistrement append-only de chaque étape d'une transaction. Ne peut jamais être modifié ni supprimé. Utilisé pour la traçabilité légale.
Compensation	Opération de correction automatique déclenchée si le crédit d'un TRANSFER échoue après que le débit a réussi. Remet le solde source à son état initial.
Bounded Context	Périmètre fonctionnel cohérent du DDD. Chaque BC possède son propre modèle, son propre schéma MySQL et son propre microservice.
Aggregate	Unité de cohérence transactionnelle. Client , Account et BankTransaction sont les trois agrégats racines du système.

4. Bounded Contexts

L'analyse DDD identifie trois bounded contexts distincts, avec des cycles de changement et des modèles de données qui ne se recoupent pas.



Pourquoi trois contextes séparés ?

Critère	Identity	Account	Payment
Fréquence de changement	Faible	Moyenne	Élevée
Charge attendue	Faible	Moyenne	Très élevée
Modèle de données	Profil client	Solde, type	Transactions, audit
Scalabilité ciblée	Non requise	Moderée	Critique (réplicable seul)

Le payment-service est le plus sollicité et le plus critique. Il peut être répliqué indépendamment sans toucher aux autres services (ADR-001).

5. Modèle de domaine

5.1 Diagramme de classes

@startuml CanBankX – Modèle de domaine

```
package "Bounded Context : Identité" {
  class Client <<Aggregate Root>> {
    +UUID id
    +String firstName
    +String lastName
    +String email <<UNIQUE>>
    +String passwordHash
    +String nas <<UNIQUE, 9 digits>>
    +String phoneNumber
    +String address
    +Status status
    +LocalDateTime createdAt
    --
    +activate()
    +suspend()
    +verifyOtp(otp)
  }

  enum Status {
    PENDING
    ACTIVE
    SUSPENDED
  }

  Client --> Status
}

package "Bounded Context : Comptes" {
  class Account <<Aggregate Root>> {
    +UUID id
    +String accountNumber <<UNIQUE, generated>>
    +UUID clientId
    +AccountType type
    +BigDecimal balance
    +LocalDateTime createdAt
    --
    +debit(amount)
    +credit(amount)
  }

  enum AccountType {
    CHECKING
    SAVINGS
  }

  Account --> AccountType
}

package "Bounded Context : Paiements" {
  class BankTransaction <<Aggregate Root>> {
    +UUID id
    +String idempotencyKey <<UNIQUE>>
    +String sourceAccountNumber
    +String targetAccountNumber
```

```

+BigDecimal amount
+TransactionType type
+TransactionStatus status
+LocalDateTime createdAt
--
+complete()
+fail()
}

class AuditLog <<Value Object>> {
+UUID id
+UUID transactionId
+String action
+String details
+LocalDateTime createdAt
}

enum TransactionType { DEBIT CREDIT TRANSFER }
enum TransactionStatus { PENDING COMPLETED FAILED }

BankTransaction --> TransactionType
BankTransaction --> TransactionStatus
BankTransaction "1" *-- "1..*" AuditLog : génère (append-only)
}

Client "1" -- "0..*" Account : possède (via clientId)
Account "1" -- "0..*" BankTransaction : impliqué (via accountNumber)

note right of AuditLog
append-only : jamais UPDATE ni DELETE
Traçabilité légale de chaque étape
end note

note right of BankTransaction
idempotencyKey : UNIQUE en DB
+ cache Redis TTL 24h
Double filet exactly-once
end note
@enduml

```

(Source : [docs/ClassDiagram.puml](#))

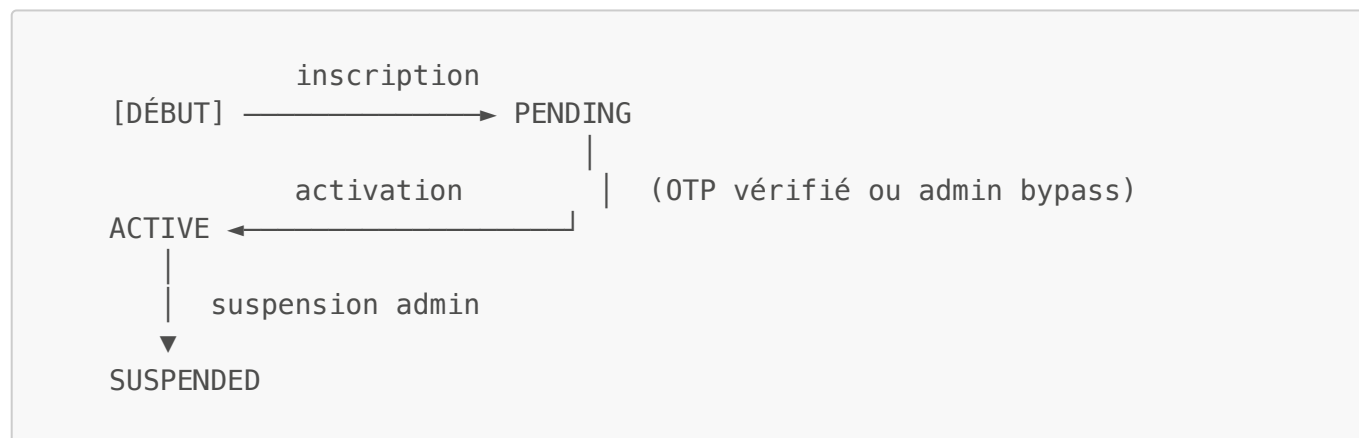
5.2 Patterns DDD utilisés

Pattern	Où	Rôle
Aggregate Root	Client , Account , BankTransaction	Point d'entrée unique pour modifier l'état. Les repositories ne retournent que des agrégats racines.

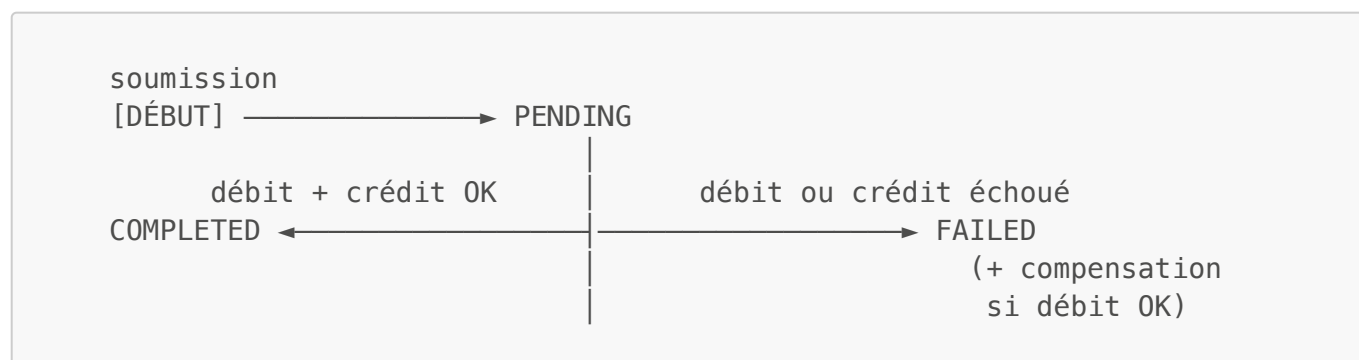
Pattern	Où	Rôle
Value Object	<code>AuditLog</code> , <code>Status</code> , <code>AccountType</code> , <code>TransactionType</code>	Immutable, définis par leurs attributs, pas par leur identité.
Repository	<code>ClientRepository</code> , <code>AccountRepository</code> , <code>BankTransactionRepository</code> , <code>AuditLogRepository</code>	Abstraction de l'accès aux données. Interface JPA, implémentation MySQL.
Domain Service	<code>PaymentService</code>	Orchestre l'opération de virement qui implique deux agrégats (<code>Account</code> source et destination). Pas de logique métier dans l'agrégat lui-même.
Anti-Corruption Layer	<code>AccountClient</code> , <code>IdentityClient</code>	Isole payment-service des modèles d'identity-service et account-service. Traduit les réponses HTTP en objets du domaine payment.
Shared Kernel	module <code>common</code>	<code>ErrorResponse</code> , <code>GlobalExceptionHandler</code> — partagés entre les 3 services, jamais modifiés unilatéralement.

5.3 Transitions d'état

Cycle de vie d'un Client :



Cycle de vie d'une BankTransaction :



6. Règles métier

Ces règles sont appliquées dans les couches service et validées par les tests k6 (smoke test — 7 cas d'erreur).

Inscription (UC-01)

Règle	Implémentation	Code HTTP
L'email doit être unique	Contrainte UNIQUE MySQL + vérification avant insert	409
Le NAS doit être unique et avoir 9 chiffres	Contrainte UNIQUE MySQL + @Size(min=9, max=9)	409 / 400
Le mot de passe est hashé avant persistance	BCrypt force 8 dans ClientService	—
Un compte PENDING ne peut pas s'authentifier	Vérification du statut dans AuthController	401

Authentification MFA (UC-02)

Règle	Implémentation	Code HTTP
Le Challenge Token expire après 5 minutes	Redis TTL 5 min sur mfa:challenge:{token}	401
Un OTP est à usage unique	Suppression de la clé Redis après validation	401
La session est stateless	SessionCreationPolicy.STATELESS dans Spring Security	—

Comptes (UC-03, UC-04)

Règle	Implémentation	Code HTTP
Le clientId doit référencer un client existant	Vérification via appel REST vers identity-service	404
Le solde ne peut pas être négatif	CHECK implicite dans AccountService.debit() avec @Transactional	422
Le débit et le crédit sont atomiques	@Transactional + UPDATE ... WHERE balance >= amount	422

Paiements (UC-05)

Règle	Implémentation	Code HTTP
-------	----------------	-----------

Règle	Implémentation	Code HTTP
Une transaction ne peut pas être dupliquée	Double filet Redis + contrainte UNIQUE SQL	201 (idempotent)
Le solde source doit être suffisant	UPDATE balance WHERE balance >= amount (1 ligne affectée sinon 422)	422
Un TRANSFER compensé passe en FAILED	Logique de compensation dans PaymentService.executeTransfer()	201 / 500
Chaque étape est tracée	AuditLogRepository.save() à chaque transition	—
L'email de confirmation n'est pas bloquant	CompletableFuture.runAsync() dans EmailService	—

7. Cas d'utilisation détaillés

UC-01 — Inscription et vérification KYC

Acteur : Client (futur), Administrateur **Service** : identity-service **Précondition** : aucune

Scénario principal :

1. Le client envoie **POST /api/clients** avec ses informations personnelles.
2. Le système valide les champs (email format, NAS 9 chiffres, mot de passe non vide).
3. Le système vérifie l'unicité de l'email et du NAS en base.
4. Le système hache le mot de passe (BCrypt force 8).
5. Le système crée le compte en statut **PENDING**.
6. Le système génère un OTP à 6 chiffres et l'envoie par email (via MailHog en dev).
7. Le client reçoit **201 Created** avec son **id**.
8. Le client soumet **POST /api/clients/{id}/verify** avec l'OTP reçu.
9. Le système valide l'OTP, passe le compte à **ACTIVE**.
10. **200 OK** retourné avec le statut mis à jour.

Scénario alternatif — Activation admin (bypass OTP) :

- L'administrateur appelle **POST /api/clients/{id}/activate**.
- Utile pour les tests automatisés k6 où lire l'email MailHog est possible mais lent.

Exceptions :

Condition	Réponse
Email déjà utilisé	409 Conflict
NAS déjà utilisé	409 Conflict
Champ manquant ou invalide	400 Bad Request
OTP invalide ou expiré	400 Bad Request

UC-02 — Authentification à deux facteurs (MFA)

Acteur : Client **Service :** identity-service **Précondition :** compte en statut **ACTIVE**

Étape 1 — Login :

1. Le client envoie **POST /api/auth/login** avec email + mot de passe.
2. Le système charge le client depuis la base et vérifie le mot de passe avec BCrypt.
3. Le système génère un **challengeToken** UUID aléatoire.
4. Le système enregistre **mfa:challenge:{challengeToken}** dans Redis (TTL 5 min).
5. Le système génère un OTP à 6 chiffres et l'envoie par email.
6. **200 OK** retourné avec **{ "challengeToken": "..."}.**

Étape 2 — Vérification MFA :

1. Le client envoie **POST /api/auth/mfa** avec **challengeToken + otpCode**.
2. Le système récupère et vérifie le Challenge Token dans Redis.
3. Le système vérifie le code OTP.
4. Le système supprime la clé Redis (token à usage unique).
5. **200 OK** retourné avec **{ "message": "SUCCESS" }.**

Exceptions :

Condition	Réponse
Mauvais mot de passe	401 Unauthorized
Compte PENDING ou SUSPENDED	401 Unauthorized
Challenge Token expiré (> 5 min)	401 Unauthorized
OTP incorrect	401 Unauthorized

UC-03 — Ouverture d'un compte bancaire

Acteur : Client **Service :** account-service **Précondition :** compte client **ACTIVE**

Scénario principal :

1. Le client envoie **POST /api/accounts** avec **clientId, accountType, initialDeposit**.
2. Le système génère un **accountNumber** unique.
3. Le système persiste le compte avec le solde initial.
4. **201 Created** retourné avec **{ id, accountNumber, type, balance }.**

Exceptions :

Condition	Réponse
clientId inexistant	404 Not Found
Type de compte invalide	400 Bad Request

UC-04 — Consultation du solde et de l'historique

Acteur : Client **Service** : account-service + payment-service (agrégation KrakenD) **Précondition** : au moins un compte ouvert

Scénario principal :

1. Le client envoie `GET /api/accounts/{accountNumber}/summary`.
2. KrakenD exécute deux appels **en parallèle** :
 - account-service : `GET /accountservice/accounts/number/{accountNumber}`
 - payment-service : `GET /paymentservice/transactions/account/{accountNumber}/recent`
3. KrakenD fusionne les deux réponses en un objet JSON unique.
4. `200 OK` retourné avec la structure agrégée.

Réponse type :

```
{
  "account": {
    "id": "...",
    "accountNumber": "ACC-1234567890",
    "type": "CHECKING",
    "balance": 450.00
  },
  "recentTransactions": [
    { "id": "...", "amount": 50.00, "type": "DEBIT", "status": "COMPLETED" }
  ]
}
```

Exceptions :

Condition	Réponse
Compte inexistant	<code>404 Not Found</code>

UC-05 — Virement bancaire avec garantie exactly-once

Acteur : Client **Service** : payment-service (orchestrateur) **Précondition** : deux comptes existants, solde suffisant sur le compte source

Scénario principal — TRANSFER :

1. Le client envoie `POST /api/transactions` avec header `Idempotency-Key: {uuid}`.
2. payment-service vérifie si la clé existe dans Redis → **miss**, continuer.
3. payment-service vérifie en base (`findByIdempotencyKey`) → **miss**, continuer.
4. payment-service insère la transaction en **PENDING** + crée un **AuditLog**.
5. payment-service appelle account-service `PATCH /debit` sur le compte source.

6. account-service exécute `UPDATE balance = balance - amount WHERE balance >= amount`.
7. payment-service appelle account-service `PATCH /credit` sur le compte destination.
8. payment-service met à jour la transaction en `COMPLETED` + AuditLog.
9. payment-service enregistre la clé dans Redis (TTL 24h).
10. `201 Created` retourné avec `{ id, status: COMPLETED, ... }`.
11. Email de confirmation envoyé de façon asynchrone (non bloquant).

Scénario alternatif — Retry idempotent :

1. Le client renvoie la même requête avec le même `Idempotency-Key`.
2. payment-service trouve la clé dans Redis → **hit**.
3. payment-service charge la transaction existante depuis la base.
4. `201 Created` retourné avec **les mêmes données** qu'à l'étape 10 ci-dessus.
5. Aucune écriture en base, aucun appel à account-service.

Scénario alternatif — Panne partielle (débit OK, crédit échoue) :

1. Étapes 1 à 6 se déroulent normalement.
2. L'appel `PATCH /credit` retourne une erreur (compte destination inexistant ou service indisponible).
3. payment-service émet un CREDIT de compensation vers le compte source (restaure le solde).
4. payment-service met à jour la transaction en `FAILED` + AuditLog avec détail de la compensation.
5. `201 Created` retourné avec `{ status: FAILED }`.

Exceptions :

Condition	Réponse
Solde insuffisant	422 Unprocessable Entity
Compte source inexistant	404 Not Found
Compte destination inexistant (TRANSFER)	404 Not Found